Fixed-point Arithmetic

Design Flow

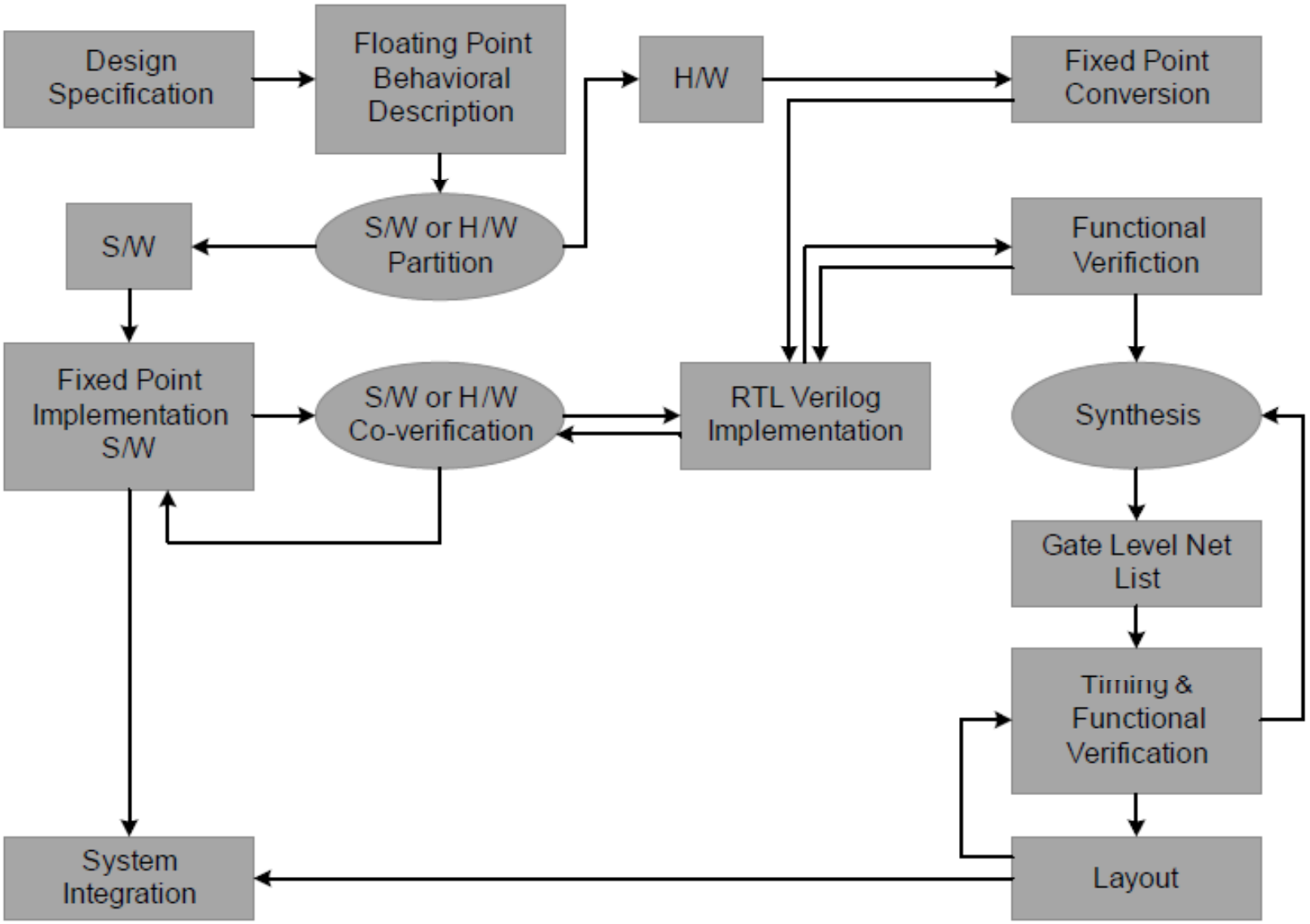


Figure 3.1 System level design components

Two Types of arithmetic

- Floating point numbers
- Fixed point numbers

Floating Point Arithmetic

- After each arithmetic operation numbers are normalized
- Used where precision and dynamic range are important
- Most algorithms are developed in FP, Ease of coding
- More Cost (Area, Speed, Power)

Fixed Point Arithmetic

- Place of decimal is fixed
- Simpler HW, low power, less silicon
- Converting FP simulation to Fixed-point simulation is time consuming
- Multiplication doubles the number of bits,
- NxN multiplier produces 2N bits
- The code is less readable, need to worry about overflow and scaling issues

Guidelines for Coding Algorithms in Matlab

- Signal processing applications are mostly developed in Matlab
- As the Matlab code is to be mapped in HW and SW so adhering to coding guidelines is critical
  - The code must be designed to work for processing of data in chunks
  - The code should be structured in distinct components
    - Well defined interfaces in terms of input and output arguments and internal data storages
  - All variables and constants should be defined in data structures
  - User defined configurations in one structure
  - System design constants in another structure
  - Internal states for each block in another structure
  - Initialization in the start of simulation

Representation of Numbers

- Types of Representation
  - one's complement
  - sign magnitude
  - canonic sign digit (CSD)
  - two's complement
  - IEEE floating point single and double precision

2's Complement Arithmetic

- In two's complement representation of a signed number a=aN-1 . . . a2 a1 a0, the MSB aN-1 represents the sign of the number
- For positive number
- For negative number
- For both

$$\boldsymbol{a} = \sum_{i\ 0}^{N\ \ 2} a_i\, 2^i \quad \text{for } a \geq 0$$

$$\boldsymbol{a} = -2^{N\ \ 1} + \sum_{i\ 0}^{N\ \ 2} a_i 2^i \quad \text{for } a < 0$$

$$\boldsymbol{a} = -2^{N\ \ 1} a_{N\ \ 1} + \sum_{i\ 0}^{N\ \ 2} a_i 2^i$$

Scaling and sign extension

- Scaling to Avoid Overflows
- Overflow adds an error that equals to the complete dynamic range of the number
- To avoid overflow, numbers are scaled down before mathematical operations

Sign Extension

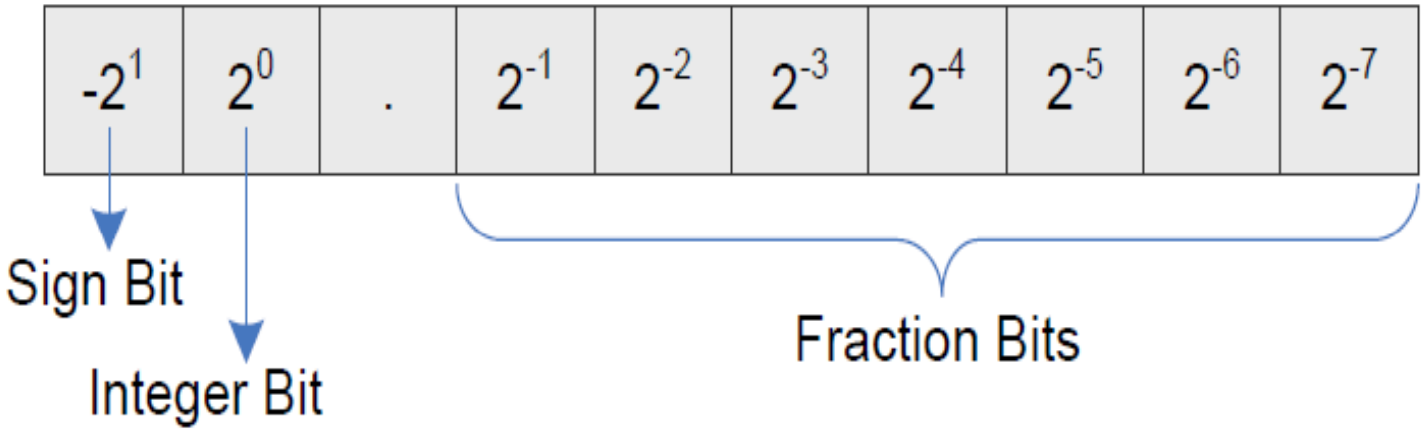
- An N bit number is extended to an M bit number M > N, by replicating M-N sign bits to the most significant bit positions
- Positive number: M-N extended bits are filled with 0s
  - The number unsigned value remains the same
- Negative number: M-N extended bits are filled with 1s,
  - Signed value remains the same
  - Equivalent unsigned value is changed

Dropping Redundant Sign bits

- When a number has redundant sign bits, these redundant bits can be dropped
- This dropping of bits does not affect the value of the number
- Example:

Qn.m Format for Fixed-point Arithmetic

- Qn.m format is a fixed positional number system for representing floating-point numbers
- A Qn.m format N-bit binary number assumes n bits to the left and m bits to the right of the binary point



Qn.m Format for Fixed-point Arithmetic

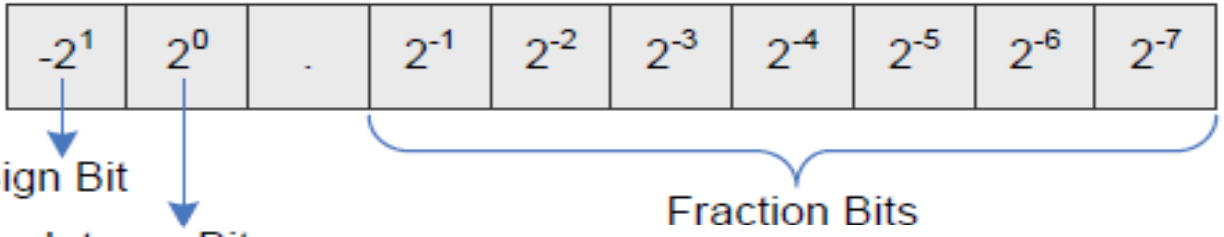
- For a positive fixed-point number, MSB is 0

$$b = 0b_{n-2} \ldots b_1 b_0 . b_{-1} b_{-2} \ldots b_{-m}$$

- For Negative numbers, MSB has negative weight and equivalent value is

$$b = -b_{n-1} 2^{n-1} + b_{n-2} 2^{n-2} + \cdots\cdots + b_1 2^1 + b_0 + b_{-1} 2^{-1} + b_{-2} 2^{-2} + \cdots\cdots + b_{-m} 2^{-m}$$

Example



01 1101 0000

In Q2.8 format, the number is :

$1 + 1/2 + 1/2^2 + 1/2^4 = 1.8125$

Equivalent fixed-point integer value is

0x1D0

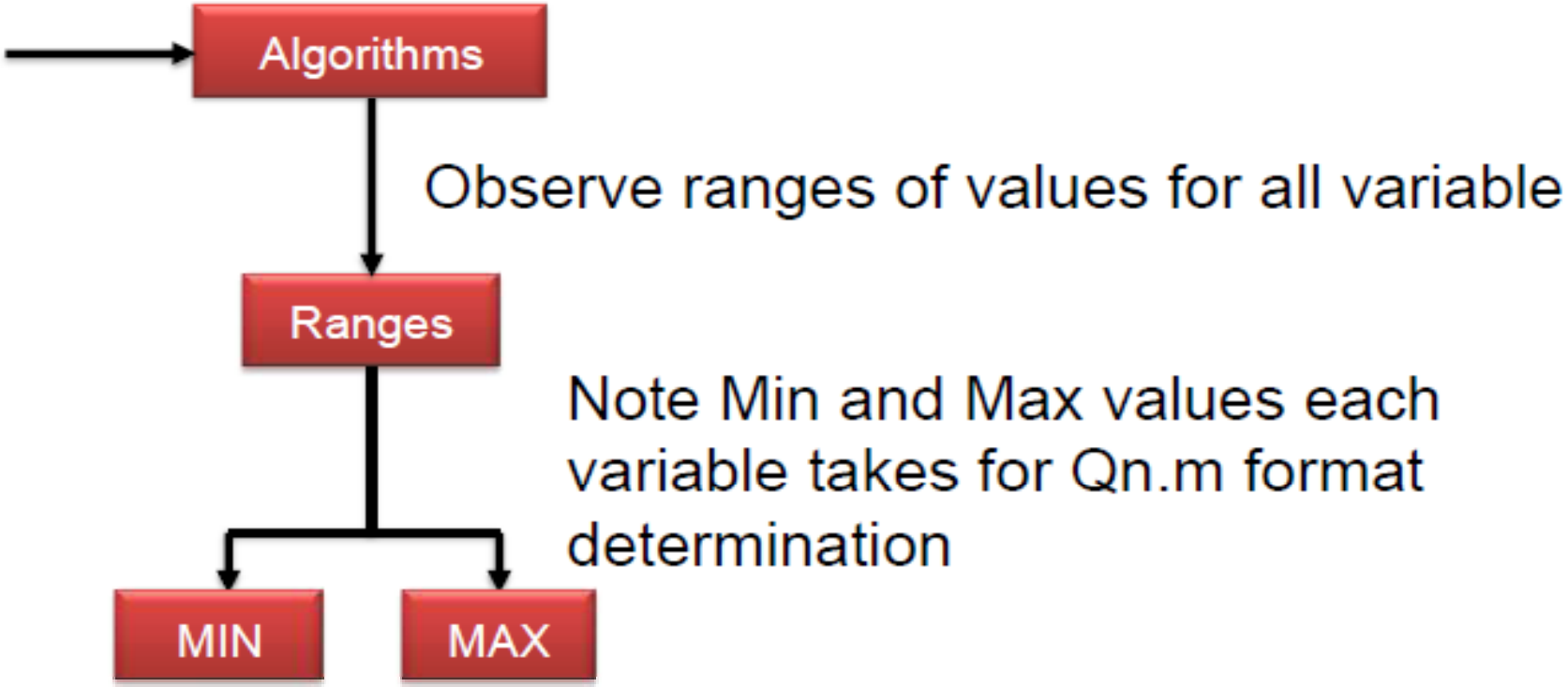
2 bits for the integer and remaining 8 bits keeps the fraction part

A 10 bit Q2.8 signed number covers -2 to +1. 9922.

Increasing the fractional bits increases the precision.

Range Determination for Qn.m Format

Run simulations for all set of inputs



Floating-point to Fixed-point Conversion

- Shift left m fractional bits to the integer part and truncate or round the results
- In Matlab, the conversion is done as

$$num\_fixed = fix(num\_float * 2^m) \text{ or}$$

$$num\_fixed = round(num\_float * 2^m)$$

- Saturate if number is greater than the maximum positive number or less than the minimum negative number.

```
if (num_fixed > 2^(N-1) –1)
        num_fixed = 2^(N-1) –1
else if (num_fixed < -2^(N-1))
        num_fixed = -2^(N-1)
```

Matlab Support for Fixed-Point Arithmetic

```
>> PI = fi(pi, 1, 3+5, 5);
>> bin(PI)
01100101
>> double(PI)
3.1563
```

Arithmetic: Addition in Q Format

- Addition of two fixed-point numbers a and b of Qn1.m1 and Qn2.m2 formats, respectively, results in a Qn.m format number, where n is the larger of n1 and n2 and m is the larger of m1 and m2.
- Example: 1111.10+0111.0110=?

Multiplication in Q Format

- Multiplications of two numbers in Qn1.m1 and Qn2.m2 formats generates product in Q(n1 + n2).(m1 + m2) format
- Four types of Fractional Multiplication:
  - Unsigned x Unsigned
  - Unsigned x Signed
  - Signed x Unsigned
  - Signed x Signed
    - Corner case
- A digital circuit covering all these cases is also required

Unsigned by Unsigned

- Multiplication of two unsigned numbers in Q2.2 and Q2.2 formats results in a Q4.4 format number
- no sign extension of partial products is required. The partial products are simply added.
- Each partial product is sequentially generated and successively shifted by one place to the left
- Example: 11.01 x 10.11

Signed by Unsigned

- Sign extension of each partial product is necessary in signed-unsigned multiplication
- Example: 11.01 x 01.01

Unsigned by Signed

- As all the partial products except the final one are unsigned, no sign extension is required except for the last one.
- For the last partial product the 2's complement of the multiplicand is computed as it is produced by multiplication with the MSB of the multiplier, which has negative weight.
- Example: 10.01 x 11.01

Signed by Signed

- Sign extend all partial products.
- Takes 2's complement of the last partial product if multiplier is a negative number.
- The MSB of the product is a redundant sign bit
- Remove the bit by shifting the product to left, the product is in $Q_{(n1 + n2 - 1) . (m1 + m2 + 1)}$
- Example: 1.10 x 0.10=?
- 11.01 x 1.101=?

Corner Case

- Exists in Signed-Signed Fractional Multiplication
- -1.0x-1.0 = -1.0 in Q fractional format is a corner case, it should be checked and result should be saturated as max positive number
- Example

Unified Multiplier

- A single signed x signed multiplier is used for four types of multiplications
- This is achieved by appending one extra bit to the left of the MSB of both of the operands
- When an operand is unsigned, the extended bit is zero, otherwise the extended bit is the sign bit of the operand

```
unsigned by unsigned: {1'b0, a}, {1'b0, b}
unsigned by signed: {1'b0, a}, {b[15], b}
signed by unsigned: {a[15], a}, {1'b0, b}
signed by signed: {a[15], a}, {b[15], b}.
```

Unified Multiplier- 1 bit extension

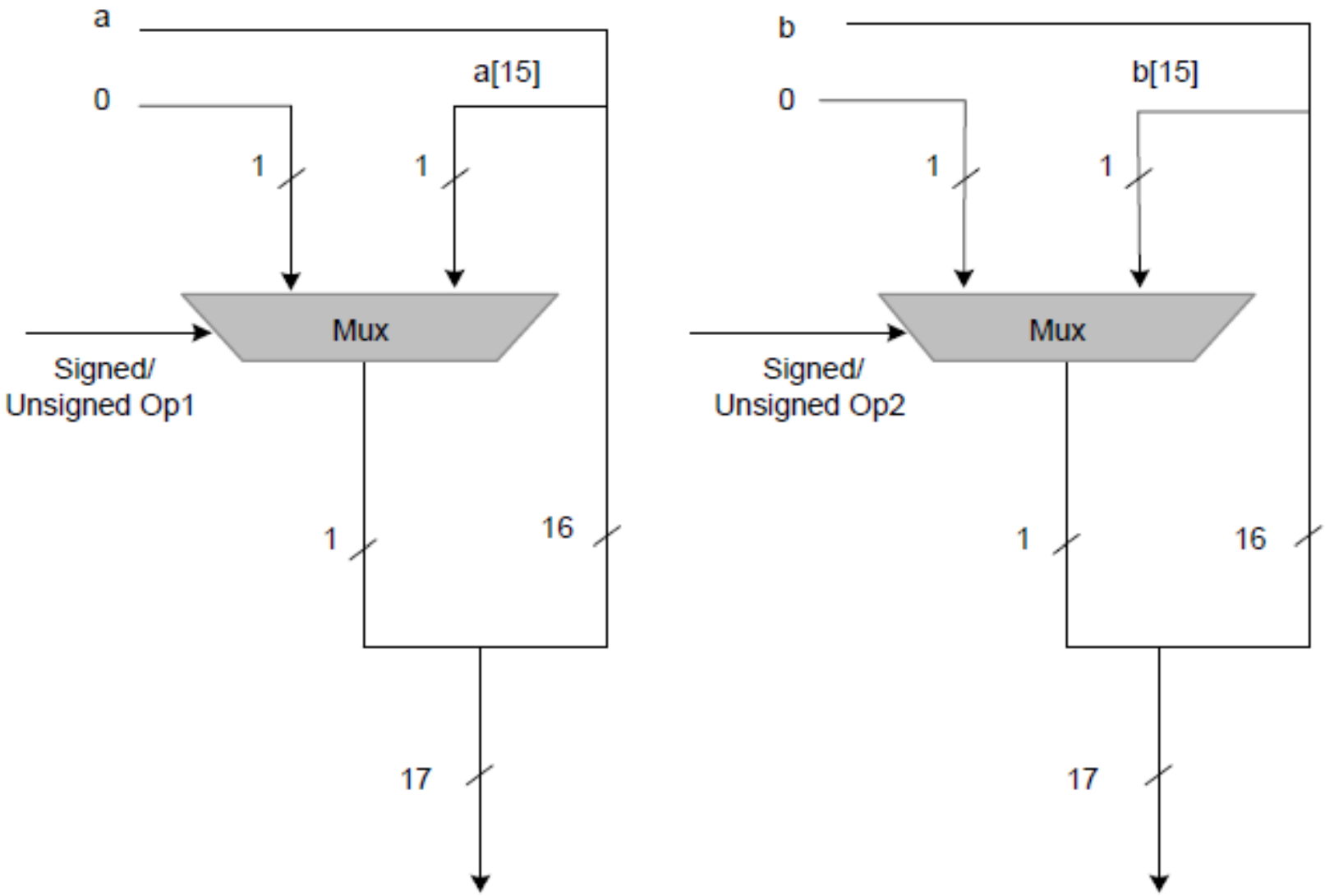
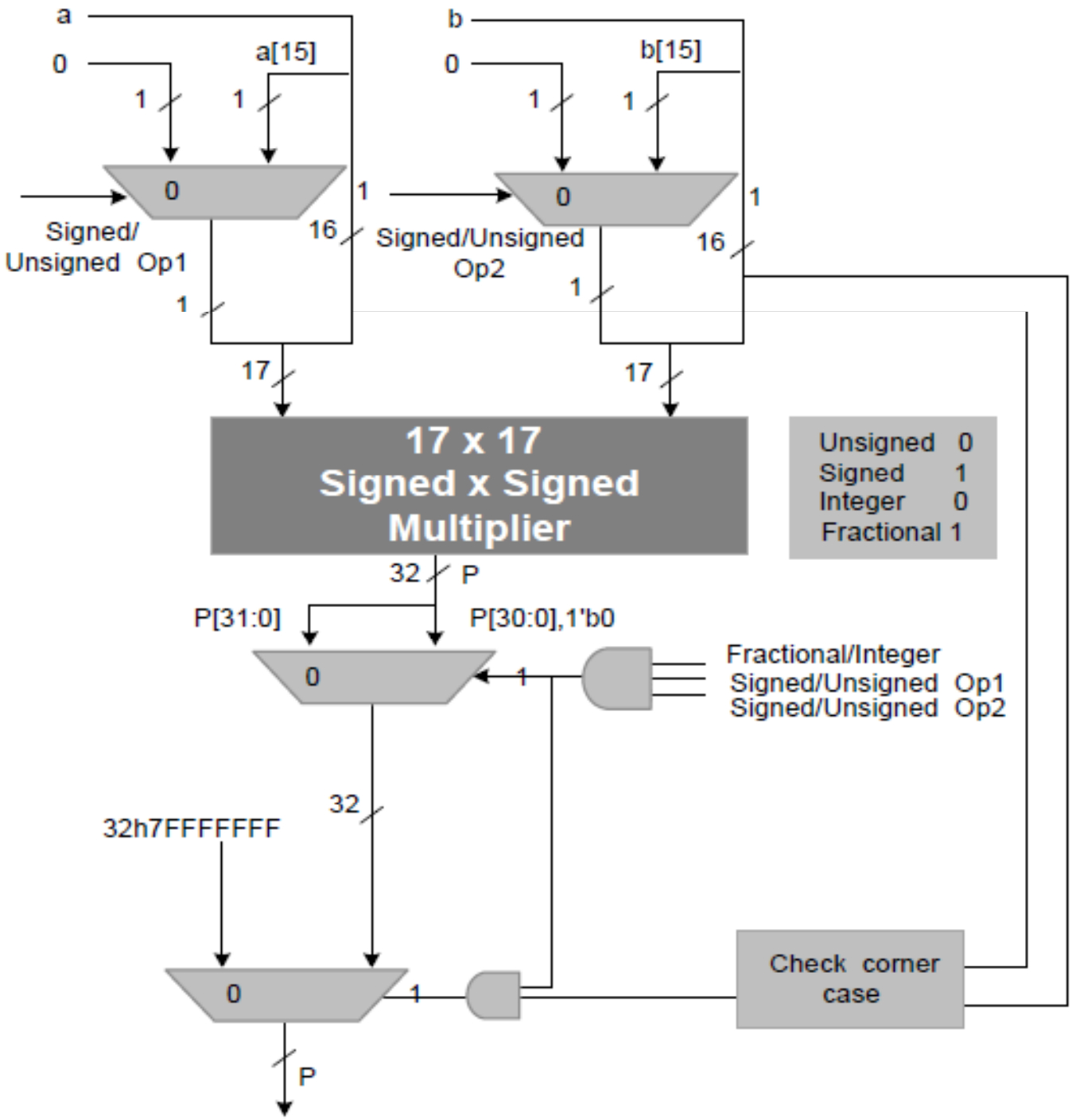


Figure 3.17 One bit extension to cater for all four types of multiplications using a single signed by signed multiplier

Unified multiplier



Bit Growth in Fixed-point Arithmetic

- Bit growth is one of the most critical issues in fixed-point arithmetic.
  - Multiplication of an N-bit number with an M-bit number results in an (N+M)-bit number
  - In recursive system each iteration increases the width
  - y[n]= ay[n-1]+x[n]
  - Multiplication of two N1 and N2 bit numbers in Qn1.m1 and Qn2.m2 results in N1+N2 bit number
  - Q(n1+n2-1).(m1+m2+1) for SxS and Q(n1+n2).(m1+m2) for the rest
  - Where as addition results is
  - Qn.m= Q max(n1,n2).max(m1.m2)

Schwarz's inequality

- For LTI system Schwarz's inequality gives an upper bound on an output

$$|y(n)| \leq \sqrt{\sum_{n=-\infty}^{\infty} h^2(n) . \sum_{n=-\infty}^{\infty} x^2(n)}$$

Truncation

- Simple Truncation

$$0111\_0111 \text{ in } Q4.4 \text{ is } 7.4375$$
$$\text{Truncated to } Q4.2 \text{ gives } 0111\_01 = 7.25$$

- Rounding Followed by Truncation
  - 1 is added to the bit that is at the right of the position of the point of truncation
  - In above example, rounding before truncation gives 7.5

Two's complement intermediate overflow property

- All is well that ends well

(a)

1.75	Q2.2	0111	1.75
+1.25	Q2.2	0101	1.25
3.00	Q2.2	1100	-1.00

(b)

3.00	Q2.2	1100	-1.00
-1.25	Q2.2	1011	-1.25
1.75	Q2.2	0111	1.75

FIR Filter considerations

- For Q1.m format inputs the outputs will mostly fit in Q1.15 if the design can guarantee that the sum of all the coefficients of an FIR filter is 1; that is:

$$\sum_{n=0}^{L-1} h[n] = 1$$

IIR Filter

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^{N} b_k z^{-k}}{1 + \sum_{k=1}^{M} a_k z^{-k}}$$

$$y[n] = \sum_{k=0}^{N} b_k \, x[n-k] - \sum_{k=1}^{M} a_k \, y[n-k]$$

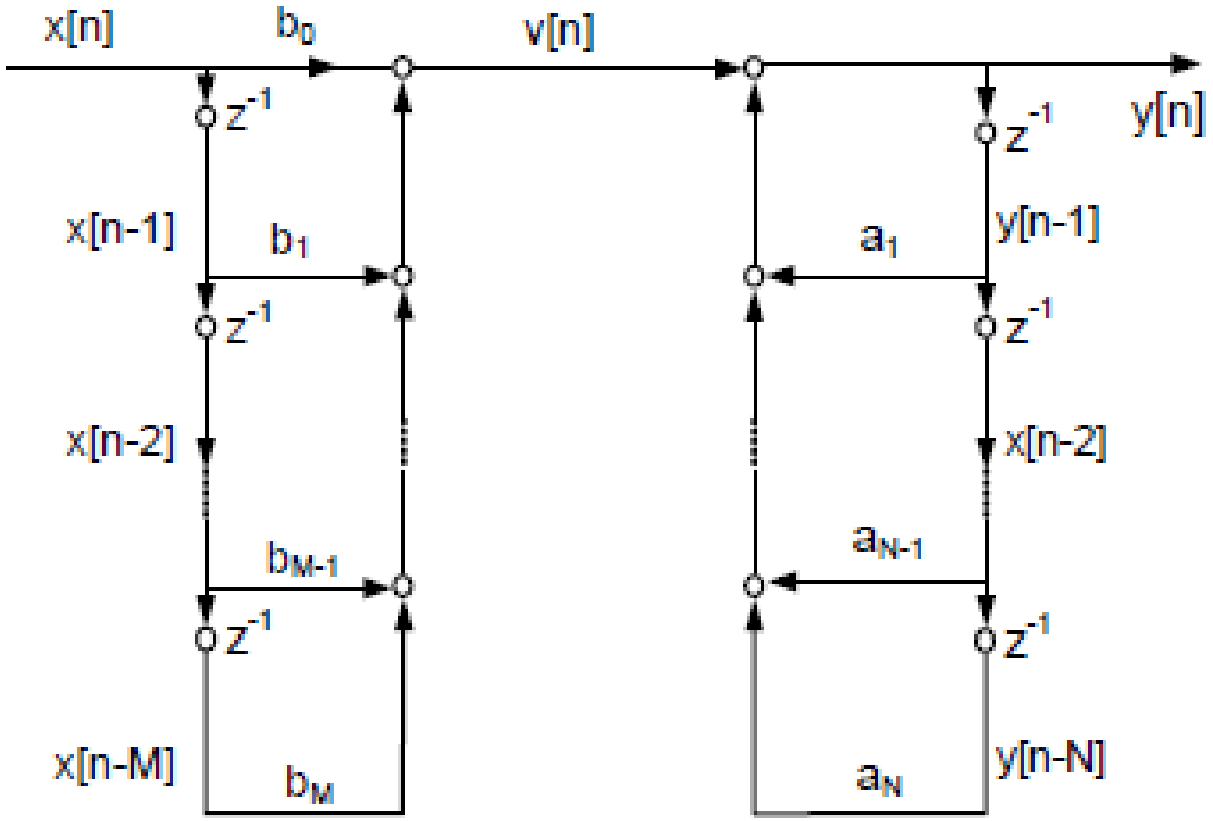
Quantization of IIR Filter Coefficients

- Fixed point conversion of double precision floating point coefficients of an IIR filter moves the pole zero location and thus affects the stability and frequency response of the system
- Example

$$\mathbf{b} = 0.0046 \quad -0.0249 \quad 0.0655 \quad -0.1096 \quad 0.1289 \quad -0.1096 \quad 0.0655 \quad -0.0249 \quad 0.0046$$
$$\mathbf{a} = 1.0000 \quad -6.9350 \quad 21.5565 \quad -39.1515 \quad 45.3884 \quad -34.3665 \quad 16.5896 \quad -4.6673 \quad 0.5860$$

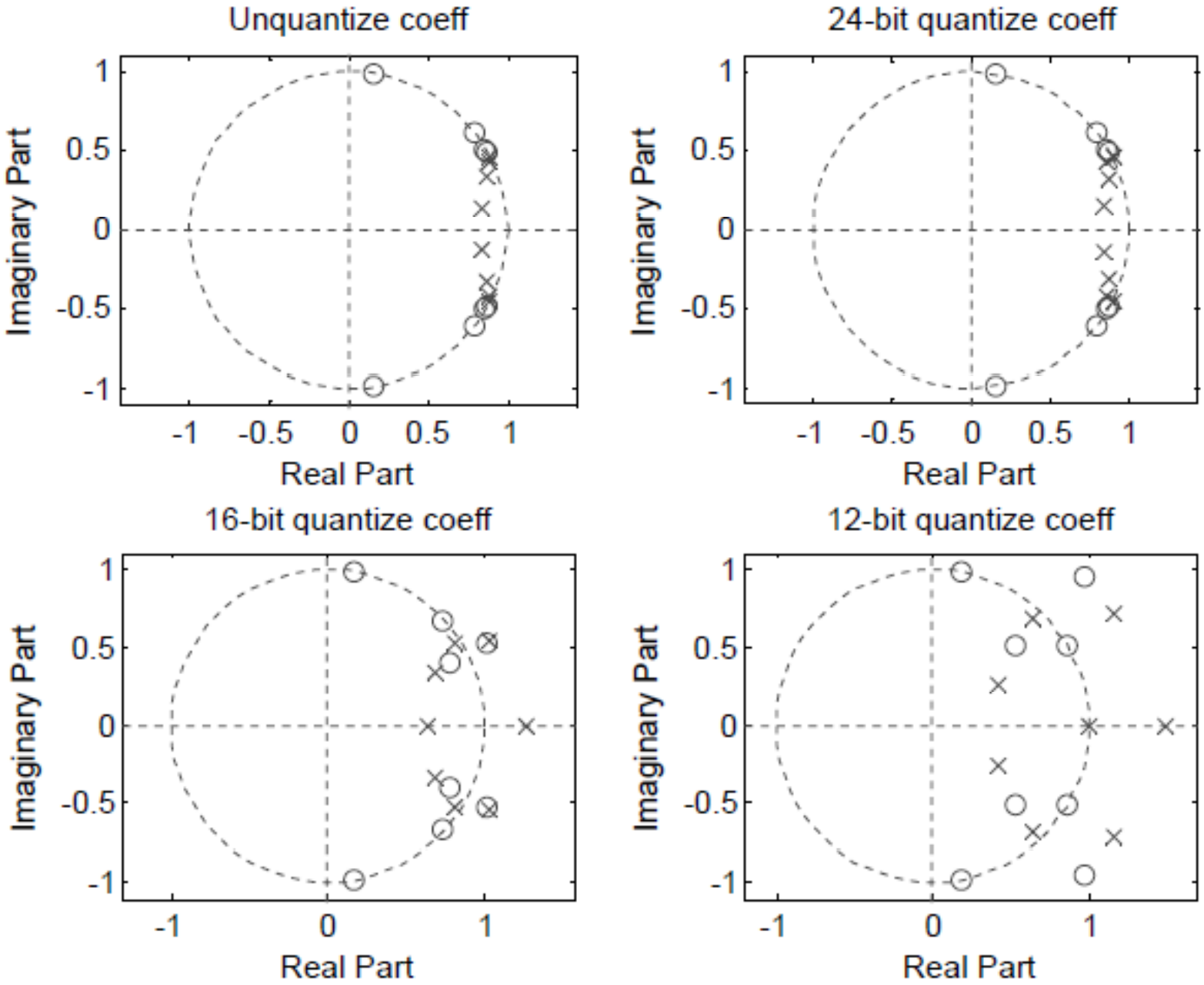
- Filter coefficients in arrays b and a are quantized to 24 bit, 16 bit and 12 bit precisions, and the Q formats of these precisions are Q1.23 and Q7.17, Q1.15 and Q7.9, and Q1.11 and Q7.6, respectively

DF-I structure

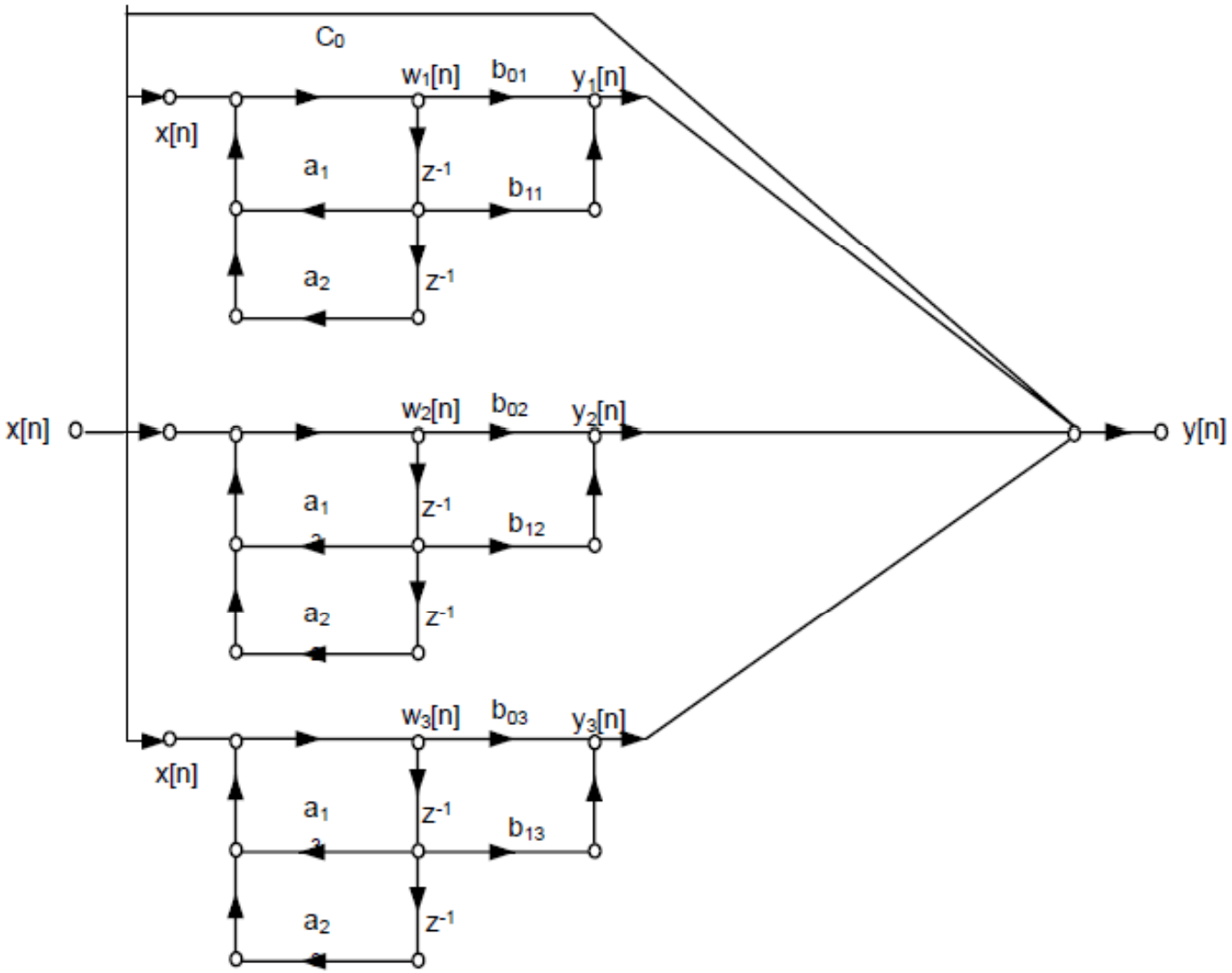


(a)

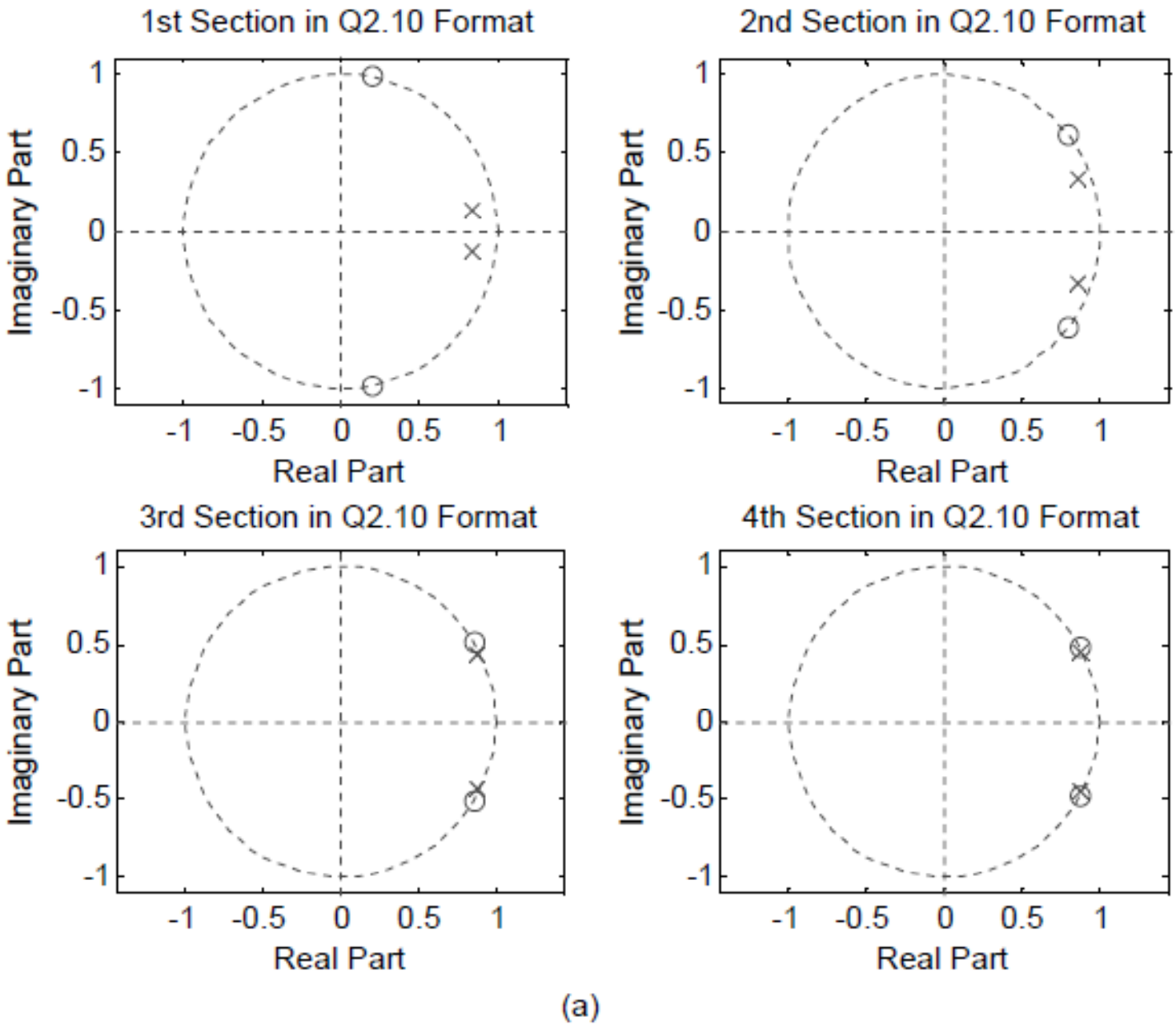
Example IIR filter



Parallel second order sections

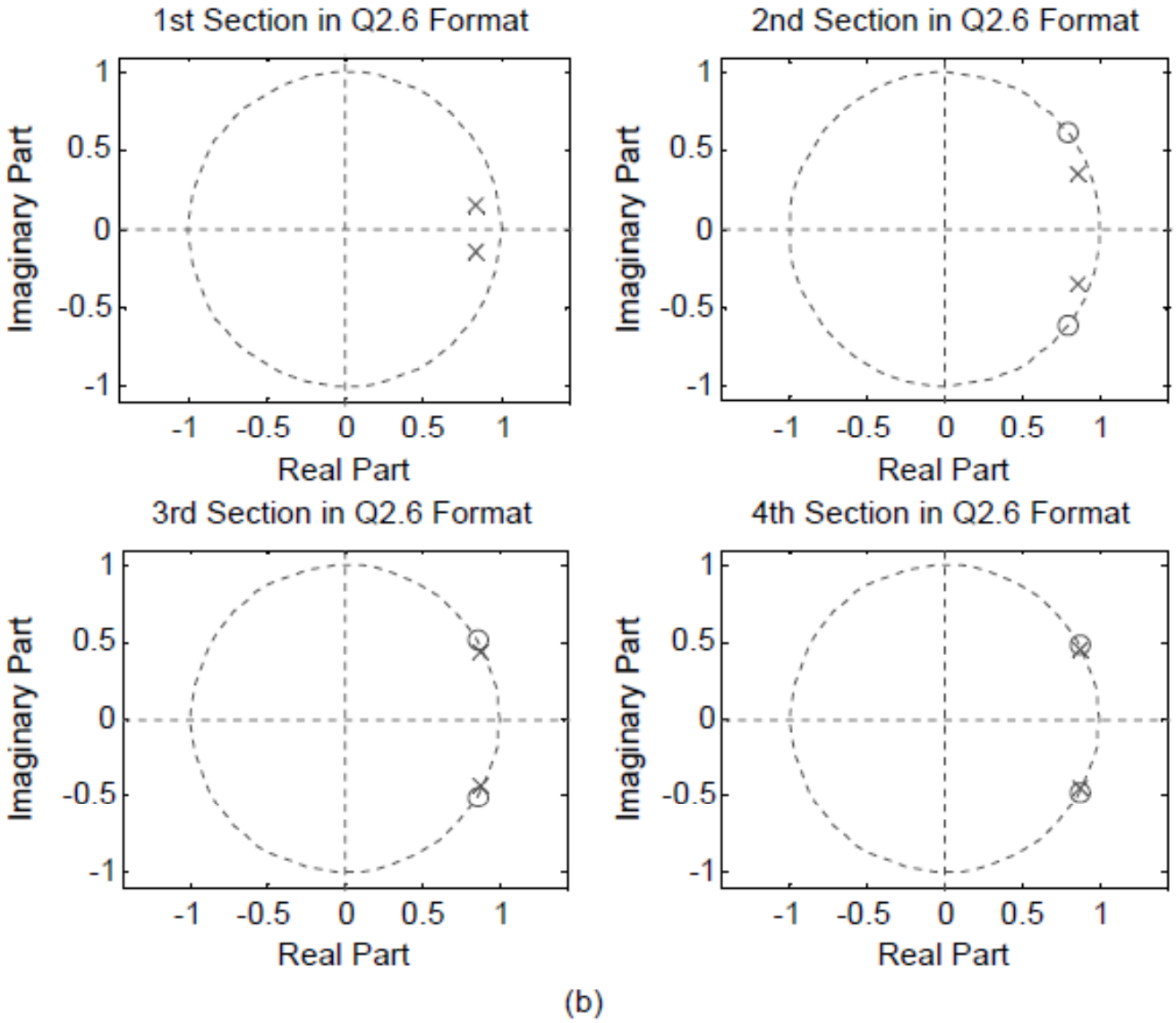


Example IIR filter



(a)

Example IIR filter



(b)

Figure 3.27 (a) Filter using cascaded SoSs with coefficients quantized in Q2.10 format. (b) Filter using cascaded SoSs with coefficients quantized in Q2.6 format

Folded FIR Filters

- symmetric or anti symmetric and impulse reponse mathematically can be represented as

$$h[M-n] = \pm h[n] \text{ for } n = 0, 1, 2, \ldots, M$$

$$y[n] = h_0 x_n + h_1 x_{n\ 1} + h_1 x_{n\ 2} + h_0 x_{n\ 3}$$

$$y[n] = h_0 (x_n + x_{n\ 3}) + h_1 (x_{n\ 1} + x_{n\ 2})$$

Folded FIR Filters

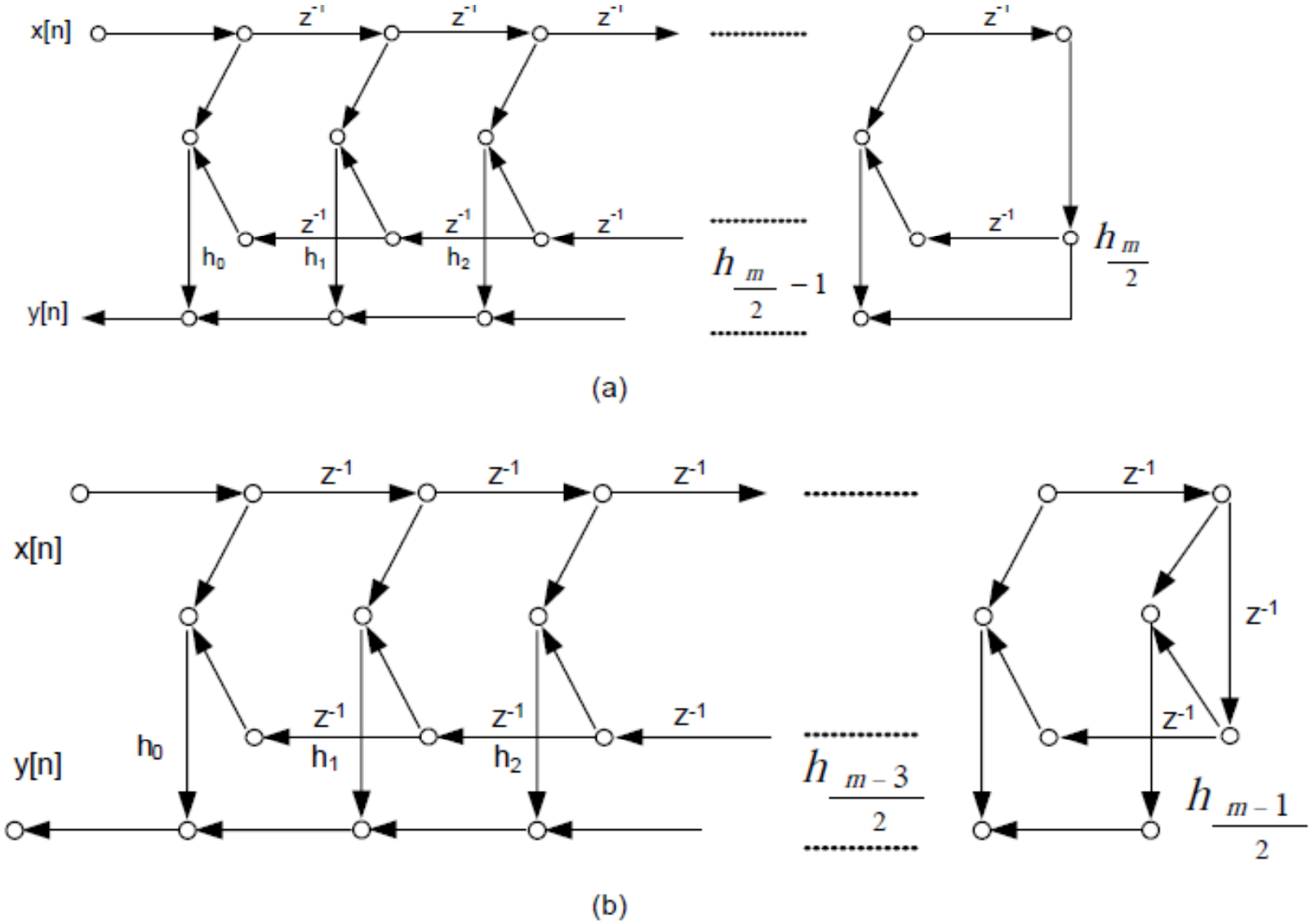


Figure 3.31 Symmetry of coefficients reduces the number of multipliers to almost half. (a) Folded design with odd number of coefficients. (b) Folded design with even number of coefficients

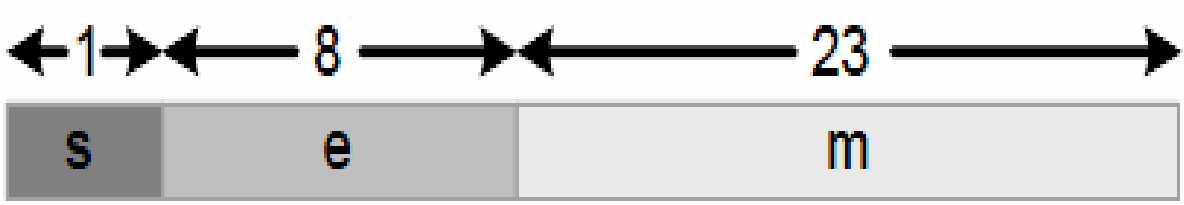
Floating Point Format

A floating point number is represented as

$$x = (-1)^s \times 1 \times m \times 2^{e-b}$$

- s represents sign of the number
- m is a fraction number >1 and < 2
- e is a biased exponent, always positive
  - The bias **b** is subtracted to get the actual exponent

IEEE single precision format

value	
$+\infty$	0_11111111_00000000000000000000
$-\infty$	1_11111111_00000000000000000000
NAN	1_11111111_10000000000000000000

IEEE single precision format

- Example
- Convert 32-bit IEEE Floating point number to decimal
- 0_10000010_11010000_00000000_0000000

The End

Q&A